

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _Pusalapati Chandu(192372119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Pchandu

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Question 1: Weak Password Hashing in Web Application

Scenario

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Simple Explanation

Think of a hash as a one-way fingerprint of a password. SHA-1 is fast, so a computer can generate billions of fingerprints per second, and because no salt (random extra text) is added, two users with the same password always get the exact same fingerprint. This means an attacker only needs to build one giant lookup table of common passwords and their fingerprints once, and it will crack every account that shares that password.

Example: If both Alice and Bob use the password "cricket123", an unsalted SHA-1 system stores the identical hash for both. An attacker who cracks Alice's hash using a rainbow table instantly knows Bob's password too, without any extra effort.

Step-by-Step Analysis

Step 1: Identify the root cause: SHA-1 is a fast, general-purpose hash not designed for password storage, so brute-forcing billions of guesses per second is cheap on modern GPUs.

Step 2: Identify the second flaw: with no per-user salt, identical passwords across different accounts produce identical hashes, letting a single precomputed rainbow table crack many accounts at once.

Step 3: Recognize the compounding effect: fast hashing plus no salting means the attacker's precomputed table investment is reused across the entire breached database instead of being wasted on one account.

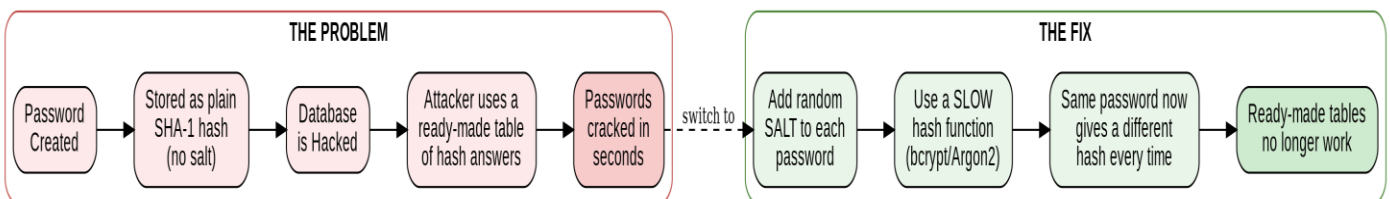
Step 4: Design the fix: replace SHA-1 with a purpose-built password hashing function (bcrypt, scrypt or Argon2id) that is deliberately slow and memory-hard, and generate a unique random salt per user stored alongside the hash.

Step 5: Understand key stretching: the chosen algorithm repeats internal hashing rounds thousands of times, so even a single guess costs meaningful CPU time, making large-scale brute-force attempts impractical.

Step 6: Plan a migration path: re-hash passwords with the new scheme the next time each user successfully logs in, since old hashes cannot be reversed to get the plaintext, and force a password reset for any accounts inactive too long to be safely migrated.

Step 7: Add defense in depth around the hashing fix: rate-limit login attempts, monitor for credential-stuffing patterns, and notify users to change reused passwords elsewhere.

Process Flowchart (Problem vs Fix)



Real-World Relevance

This exact pattern caused major real-world breaches such as the 2012 LinkedIn incident, where millions of unsalted SHA-1 password hashes were leaked and a large fraction were cracked within days using publicly available rainbow tables and GPU cracking rigs. It is precisely why security standards (OWASP, NIST) now mandate salted, slow hashing algorithms for any system that stores passwords.

Recommendation and Justification

- Recommended algorithm: Argon2id (winner of the Password Hashing Competition), or bcrypt/PBKDF2 where Argon2 is unavailable.
- Use a unique, cryptographically random salt (at least 16 bytes) per password, stored alongside the hash.
- Tune the work factor (iterations/memory cost) so hashing takes roughly 100-300ms on production hardware, balancing security against login latency.
- Enforce additional layers: rate-limiting login attempts, multi-factor authentication, and breach-monitoring to reduce impact even if hashes are later stolen.

Conclusion

Weak, unsalted hashing turns a single database breach into an avalanche of cracked accounts. Moving to a salted, slow, memory-hard hashing scheme is a small implementation change that removes the two root causes at once - it stops precomputed tables from working and makes brute-forcing each password individually far too slow to be worthwhile.

Question 2: Digital Signature Compromise in Financial System

Scenario

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Simple Explanation

A digital signature works like a unique, unforgeable seal that only one person can make, using their private key. Everyone else can check the seal using the matching public key, but only the private key holder can create it. If someone steals the private key, they can stamp that unforgeable seal on anything they want - the bank has no way to tell the difference between the real owner and the thief, because the signature verifies correctly either way.

***Example:** It is like losing the only key to a signature stamp that a bank accepts as proof of your identity. If a thief finds that stamp, every document they stamp with it will be treated by the bank as if you personally approved it, even though you never saw the transaction.*

Step-by-Step Analysis

Step 1: Understand the trust model: digital signatures prove authenticity and non-repudiation only because the signing key is assumed to be secret and solely under the legitimate user's control.

Step 2: Assess the impact on authentication: once the private key is stolen, the attacker can produce signatures that are cryptographically indistinguishable from the genuine user's, so the system will authenticate fraudulent transactions as legitimate.

Step 3: Assess the impact on non-repudiation: the compromised user can no longer be reliably held accountable for transactions signed with the stolen key, and conversely cannot easily disown a fraudulent transaction because it verifies correctly against their public key.

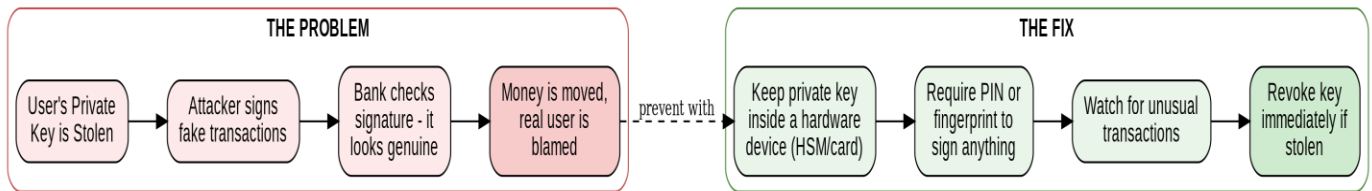
Step 4: Trace how the compromise likely happened: malware, phishing, weak key storage (e.g. plaintext key files on a laptop), or lack of hardware-backed protection for the key.

Step 5: Recognize the business impact: fraudulent transactions may be processed before anyone notices, and resolving disputes becomes difficult since the cryptographic proof itself now points at the victim.

Step 6: Design mitigations that reduce both the chance of theft and the impact if theft occurs, including detection and rapid revocation of the compromised key.

Step 7: Plan an incident response procedure: immediately revoke the compromised certificate/key, notify affected parties, and re-issue a new key pair through a verified, in-person or strongly authenticated channel.

Process Flowchart (Problem vs Fix)



Real-World Relevance

This scenario mirrors real incidents in cryptocurrency exchanges and online banking where stolen private keys led to irreversible unauthorized transfers, since blockchain and PKI-based systems trust any transaction signed with a valid key by design. It is why modern financial systems increasingly require hardware wallets or HSM-backed signing rather than software-only key storage.

Recommendation and Justification

- Store private keys in a Hardware Security Module (HSM) or smart card/USB token so the raw key material never leaves protected hardware.
- Require multi-factor confirmation (PIN, biometric, or a hardware button press) before the key can be used to sign a transaction.
- Maintain a Certificate Revocation List (CRL)/OCSP responder so a compromised key's certificate can be revoked immediately once detected.
- Apply anomaly detection on transaction patterns (unusual amount, destination, or timing) to flag suspicious signed transactions for manual review even though they verify correctly.

Conclusion

A digital signature is only as trustworthy as the secrecy of the private key behind it. Protecting that key with hardware isolation, multi-factor confirmation, and fast revocation is essential, because once the key is out, the cryptography itself cannot distinguish the thief from the rightful owner.

Question 3: Kerberos Authentication Failure in Distributed System

Scenario

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Simple Explanation

Kerberos tickets are like concert tickets stamped with an exact issue time and a short expiry window, used specifically so a stolen ticket cannot be reused later (replayed). But this only works if every gate (server) checking the ticket agrees on what time it is. If one server's clock is running fast or slow, it may look at a perfectly valid ticket and think it has already expired, or that it was issued in the future, and reject a genuine user.

***Example:** Imagine your watch is 10 minutes fast. You arrive on time for a train that departs at 10:00, but your watch reads 10:11, so you assume you already missed it and don't board - even though the train is still there. That is exactly what happens when a Kerberos server's clock drifts from the Key Distribution Center's clock.*

Step-by-Step Analysis

Step 1: Recall that Kerberos tickets embed timestamps and a limited lifetime specifically to prevent replay attacks, so the protocol inherently depends on all parties agreeing on the current time.

Step 2: Identify the failure mode: if a client or server's clock drifts too far from the Key Distribution Center's (KDC) clock, a freshly issued ticket can appear already expired, or an authenticator can appear to be a replay, causing the KDC or application server to reject valid requests.

Step 3: Note that Kerberos implementations typically allow only a small clock skew tolerance (commonly around 5 minutes) precisely because larger tolerances would weaken replay protection.

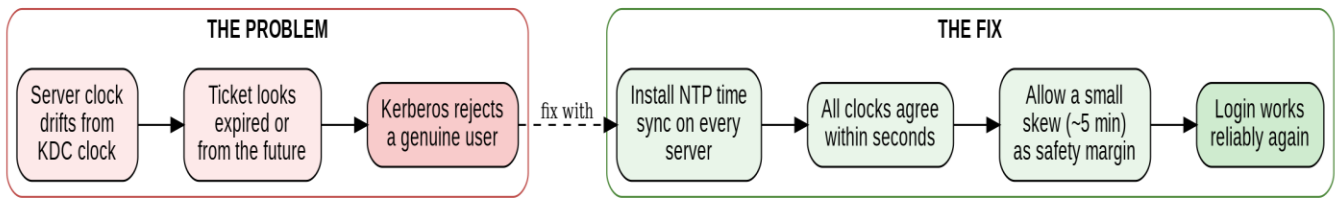
Step 4: Diagnose likely causes of drift: servers without NTP configured, virtual machines whose clocks drift after being paused/resumed, or NTP servers that are themselves unreachable or misconfigured.

Step 5: Consider the operational symptoms: authentication failures that appear random, affect some servers but not others, and correlate with server restarts or VM snapshot restores - a strong hint that the issue is clock drift rather than credentials.

Step 6: Design the fix around synchronizing time reliably across every host that participates in the Kerberos realm, and monitoring for drift before it causes failures.

Step 7: Plan ongoing maintenance: alerting when a host's drift approaches the tolerance threshold, so administrators can intervene before users experience login failures.

Process Flowchart (Problem vs Fix)



Real-World Relevance

This is a very common real-world issue in enterprise Active Directory environments (which use Kerberos internally), where a domain controller or member server with a misconfigured or unreachable NTP source causes intermittent, hard-to-diagnose login failures across the organization - usually resolved simply by fixing time synchronization.

Recommendation and Justification

- Deploy and enforce NTP (or a more precise protocol such as PTP for very tight environments) on every client, application server, and the KDC itself.
- Point all servers at the same trusted, redundant set of time sources rather than each server picking an independent source.
- Monitor clock drift proactively (alerting when drift approaches the Kerberos skew tolerance) instead of waiting for authentication failures to reveal the problem.
- Keep the Kerberos clock-skew tolerance at its default conservative value rather than widening it, since widening it to hide sync problems would reopen the replay-attack window.

Conclusion

Kerberos trades a small amount of operational fragility (dependence on synchronized clocks) for strong replay protection. The correct fix is never to loosen the time-window check, since that reintroduces the very attack it defends against - instead, invest in reliable, monitored time synchronization across the whole realm.

Question 4: Certificate Authority Breach in PKI System

Scenario

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Simple Explanation

A Certificate Authority is like a notary that stamps an official seal on a document to prove it is genuine, and everyone agrees in advance to trust that notary's seal. If someone breaks into the notary's office and starts stamping fake documents with the real seal, anyone checking those documents will still see a valid-looking seal and believe them, because the checking process only verifies the seal is authentic, not who actually applied it.

***Example:** In 2011, the DigiNotar Certificate Authority was breached, and attackers issued a fake certificate for google.com. Because DigiNotar was a trusted root CA in browsers at the time, the fake certificate looked completely valid, letting attackers intercept supposedly secure Gmail traffic for real users without any browser warning appearing.*

Step-by-Step Analysis

Step 1: Understand the trust impact: browsers and clients trust any certificate that chains up to a root CA in their trust store, so a compromised CA can mint certificates for any domain that will be accepted as genuine by every relying party.

Step 2: Identify the practical attack this enables: with a fake but validly-signed certificate for a target domain, an attacker can perform man-in-the-middle interception of supposedly secure (TLS) traffic without triggering browser warnings.

Step 3: Recognize why this is hard to detect through normal means: the fraudulent certificate passes standard chain validation because the compromised CA is a legitimately trusted root or intermediate.

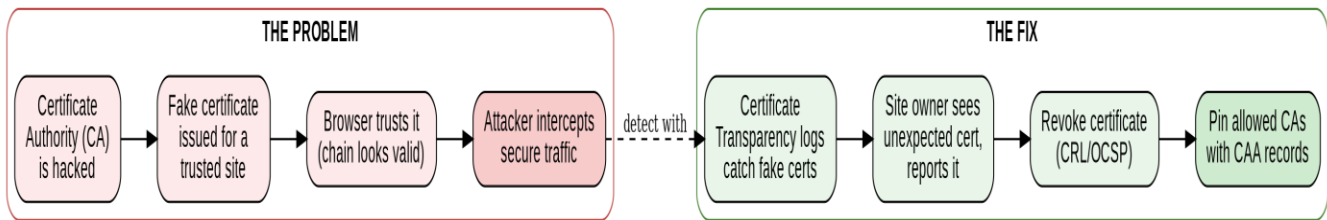
Step 4: Identify detection mechanisms designed for exactly this scenario: Certificate Transparency (CT) logs, which require CAs to publicly log every certificate they issue so unexpected certificates for a domain can be spotted by monitoring services or domain owners.

Step 5: Understand how CT monitoring works in practice: domain owners or third-party monitoring services continuously scan public CT logs for any new certificate issued for their domain, and alert immediately if one appears that was not requested.

Step 6: Identify mitigation and containment steps once a rogue certificate or compromised CA is discovered, including revocation and, in severe cases, complete removal of the CA from trust stores.

Step 7: Consider longer-term structural defenses that reduce reliance on any single CA's judgment, such as CAA DNS records and certificate pinning.

Process Flowchart (Problem vs Fix)



Real-World Relevance

Beyond DigiNotar, the Symantec CA was distrusted by Google Chrome and Mozilla Firefox in 2017-2018 after a long history of improperly issued certificates, forcing a mass migration of certificates to other CAs - showing that browser vendors are willing to remove an entire CA from the trust store as an ultimate mitigation.

Recommendation and Justification

- Detect via Certificate Transparency log monitoring - domain owners should watch CT logs for any certificate issued for their domains that they did not request.
- Use OCSP stapling and/or up-to-date CRLs so relying parties can check revocation status in near real time.
- Revoke the compromised CA's certificate (or the specific fraudulent certificates) and, in severe cases, remove the CA from browser/OS trust stores.
- Adopt Certificate/Public-Key Pinning or CAA (Certification Authority Authorization) DNS records so only specific, authorized CAs are allowed to issue certificates for a given domain, limiting the blast radius of any single CA's compromise.

Conclusion

PKI security depends entirely on every CA in the trust chain behaving correctly, so a single compromised CA can undermine trust for the whole web until it is caught. Certificate Transparency, revocation checking, and CAA records together turn an invisible, silent compromise into something that is detected quickly and contained before it causes widespread damage.

Question 5: Integrity Verification in Cloud Data Storage

Scenario

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Simple Explanation

Storing a hash of a file is like sealing a box and writing down its exact weight on a label kept in a separate drawer. To check the box has not been opened and swapped, you must actually walk over, weigh it again, and compare it to the label every single time. If nobody ever re-checks the weight, or if the label was kept inside the same box instead of a separate drawer, someone can tamper with the contents and nobody will notice.

***Example:** A cloud drive computes a hash when a file is uploaded but only ever displays that same stored hash back to the user on every download screen, without recomputing it from the actual current file. An attacker who quietly modifies the stored file on the server leaves the displayed hash unchanged, so users see a hash that matches what they expect and never realize the file itself has been altered.*

Step-by-Step Analysis

Step 1: Identify the intended integrity model: a hash of the file is computed at upload time and stored as a reference so that, at any later point, recomputing the hash and comparing it to the stored value should reveal tampering.

Step 2: Identify how implementation can defeat this model even when the underlying hash algorithm is fine: for example, never re-checking the hash on download, comparing hashes with a method vulnerable to short-circuiting, or storing the reference hash in the same insecure location as the file (allowing both to be altered together).

Step 3: Recognize a further weakness if a fast, non-cryptographic or broken hash (e.g. MD5) is used, since it becomes computationally feasible to craft a different file with the same hash value (a collision), defeating the check entirely even with correct validation logic.

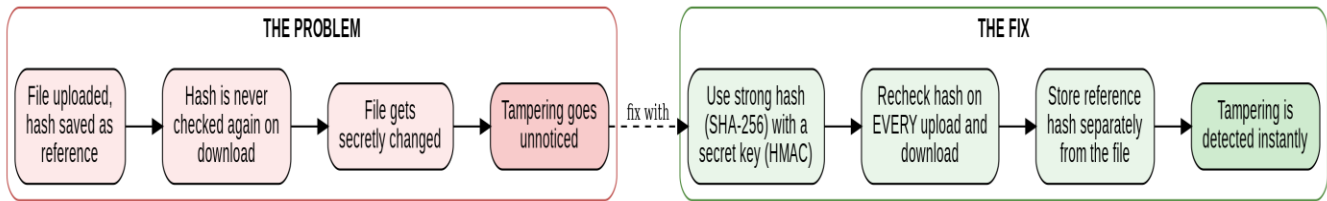
Step 4: Distinguish detection from prevention: hashing alone can only detect tampering after the fact and only if it is actually checked - it does not prove who made an authorized change, which is where signatures add value.

Step 5: Trace the specific failure reported: users noticing tampering despite hash validation being 'in place' strongly suggests the check exists in code but is either not invoked on the right path (e.g. only on upload, never on download) or compares against a reference that was stored insecurely.

Step 6: Design a corrected integrity pipeline that validates on every access, uses a strong algorithm, and protects the reference hash independently from the file.

Step 7: Plan verification for large-scale storage: periodic background integrity scrubbing that walks all stored files and re-validates their hashes, catching silent corruption or tampering even when no user happens to download the affected file.

Process Flowchart (Problem vs Fix)



Real-World Relevance

Major cloud providers such as AWS S3 and Azure Blob Storage attach content checksums (e.g. MD5/SHA-256, or dedicated ETags) to every object and support automatic integrity verification on both upload and download, precisely to prevent exactly this class of silent tampering or corruption from going unnoticed.

Recommendation and Justification

- Use a strong, collision-resistant hash such as SHA-256, and prefer HMAC-SHA256 with a secret key so an attacker who also modifies the stored reference hash still cannot forge a valid tag without the key.
- Re-verify the hash/HMAC on every download and periodically during at-rest storage (integrity scrubbing), not just once at upload time.
- Store the reference hash separately from the file (e.g. in an access-controlled metadata database or signed manifest) so tampering with the file alone cannot also silently update the expected hash.
- For large datasets, use a Merkle tree so any single altered block can be pinpointed efficiently, or use digital signatures over the hash so integrity checks also carry non-repudiable proof of the file's origin.

Conclusion

A hashing scheme is only as strong as the discipline with which it is actually checked. Computing a hash once at upload time provides no protection unless it is independently stored, kept safe from tampering itself, and re-verified every time the file is accessed - otherwise the integrity check exists on paper but does nothing in practice.